

```
test@test-machine:~/ $ ./pmq_sender_priority
Sender MQ /mq-postman opened
Sender: sending message: [Hello there - 1] with priority: [10]
Sender: sending message: [Hello there - 2] with priority: [0]
Sender: sending message: [Hello there - 3] with priority: [10]
Sender: sending message: [Hello there - 4] with priority: [0]
Sender: sending message: [Hello there - 5] with priority: [10]
Sender: sending message: [Hello there - 6] with priority: [0]
Sender: sending message: [Hello there - 7] with priority: [10]
Sender: sending message: [Hello there - 8] with priority: [0]
Sender: sending message: [Hello there - 9] with priority: [10]
Sender: Exit
```

Figure 6: A sender process sending a series of messages with alternating priority to the message queue

priority alternating between 0 and 10. The receiver process receives the highest priority (10) messages available in the queue first, and then the lower priority (0) messages.

Deleting message queues

POSIX message queues have kernel persistence. This means a queue stays in kernel space even after the process that created it exits without deleting the queue. A process can delete a message queue by calling `mq_unlink()` and passing the name of the message queue as the parameter. A user, too, can remove the message queue by giving the command `rm /dev/mqueue/<qname>`. If a process calls `mq_unlink()` with the unread messages in the queue, the said POSIX message queue can still get deleted from kernel space.

Pitfalls

One feature that differentiates POSIX message queues from System V message queues is the asynchronous notification when a new message arrives on an empty queue. When using asynchronous notification to receive messages, a process can be notified either by a signal or by invocation of a handler. A process needs to register itself to receive asynchronous notification, so that when a message arrives on an empty queue, a notification is sent to the registered process. By doing so, when a message arrives in an empty queue, the registered process is notified where the message should be read. To get notification on the arrival of another message, the process must re-register and receive asynchronous notification. This needs to be done every time a message arrives in an empty queue.

Suitable care needs to be exercised if a process sends a message in-between the `mq_receive()` and `mq_notify()`, as asynchronous notification works only when a message is received on an empty queue. If the queue already has messages and `mq_notify()` is called, the notification will never happen and, consequently, the messages will never be read out from the queue.

```
test@test-machine:~/ $ ./pmq_receiver_priority
Receiver MQ /mq-postman opened
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 1] with priority: [10]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 3] with priority: [10]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 5] with priority: [10]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 7] with priority: [10]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 9] with priority: [10]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 2] with priority: [0]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 4] with priority: [0]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 6] with priority: [0]
Receiver: blocked, waiting for messages...
Receiver: message received: [Hello there - 8] with priority: [0]
Receiver: Exit
```

Figure 7: A receiver process receiving the highest priority messages first from the message queue

If a `fork()` is named after `mq_notify()`, then the child process doesn't inherit the parent's message queue notification registration. In the event of `exec()` or process termination, message queue descriptors associated with this process are closed and the message notification registration on the said queue is de-registered. In either case, the POSIX message queue still remains in the system unless `mq_unlink()` is called, as explained earlier.

When the design requires that `mq_notify()` is used, ensure that POSIX message queue is opened in non-blocking mode.

A single POSIX message queue can be used for both sending and receiving a message between two processes. But it gets challenging to find out whether the message in the queue is something sent by the sender to be read by the receiver or is it the (response) message sent by the receiver to be read by the sender. So it is always better to make use of two different message queues, one for sending messages and one for receiving them.

I have demonstrated the advanced concepts or behaviour of the POSIX message queues with supporting source code and a screenshot of the execution console. This article will be helpful to developers/architects who intend to use POSIX message queues beyond the basic level. **END** 🐼

 By: Krishna Mohan Dani

The author is a group technical architect in the embedded platform lab of HCL Technologies, Bengaluru. His professional interests include architectural design and development of embedded software for automotive, aerospace, IoT and consumer electronics.